(continued)

accept code as being complete unless there are enough tests to go with it. I'm indifferent regarding whether the tests should be written first. You could say I'm a fan of TDD and BDD but a little flexible about the implementation.

8.1.2 Automated testing in Python

The Python standard library includes the `unittest` module to help you write automated tests. (You can write other categories of tests using it as well— don't let the name confuse you). The library is based on four concepts:

- Fixtures
- Test cases
- Test suites
- Test runners

A *fixture*, in this case, is the preparation needed to run a test and any clean-up needed when the test is finished. This terminology overlaps Django's use of the word *fixture*, which is data you can load into the database. The ideas are related, though: most of the time, test preparation is about getting your data set up.

A *test case* is the code that runs to perform your test. In the `unittest` module, these are defined as classes. A *test suite* is a collection of test cases, and a *test runner* is a program that determines which tests to run. Python's default test runner looks for files whose name begins with "test_" and runs any test cases found inside.

Django builds on top of the Python `unittest` module. It uses the same structure but provides its own test class and runner. Django's runner knows how to deal with the Django ORM, and each time you run a set of tests, it creates a new copy of a database, which you can populate for your tests. Django also subclasses the test case class providing additional methods that specialize in testing Django views.

Other test frameworks

There are several third-party libraries that are alternatives to Python's `unittest` framework. Two of the most popular are pytest (https://docs.pytest.org) and nose (https://nose.readthedocs.io/). Both of these have plugins for interacting with Django. If you're already a user of either of these, you can write your Django tests in your favorite flavor, if you wish.

8.2 Your first Django test

Django thinks tests are important. You can tell, as it automatically creates a tests.py file when a new app gets added. The skeleton version of this file contains a single import: the `django.test.TestCase` class. This class inherits from the Python class of the same name. To write a unit test, you inherit from this class and add methods with names that begin with "test_".